Universidad Andrés Bello · Facultad de Ingeniería

Informe Solemne 1

Smart Campus IoT: monitoreo ambiental y de ocupación de salas

Sistema IoT con microservicios en Raspberry Pi y plataforma en la nube

Integrantes

Zaid Isamit Alvial

Jhon Rojas Caceres

Agustin Carmona Aguilera

Repositorio: github.com/ZaidIsamit/RaspberryPi

Plataforma: raspberry-pi-y1p2.vercel.app

Introducción

En un campus universitario es útil saber si una sala está ocupada y en qué condiciones se encuentra, ya que con esa información se puede decidir cuándo climatizar, ventilar o revisar un uso fuera del horario permitido. Para este trabajo se desarrolló Smart Campus IoT, un sistema que mide la temperatura, la humedad y la presencia de personas en la Sala 101 mediante una Raspberry Pi y que muestra los datos en una plataforma web.

El objetivo general fue diseñar, implementar y desplegar un sistema IoT que capture datos reales con sensores, los procese con microservicios en contenedores, los almacene en Cloud Firestore y los presente en una plataforma web, evaluando además el consumo de recursos que genera. Como objetivos específicos se plantearon: integrar dos sensores de distinto tipo y combinar sus lecturas, dividir el software del dispositivo en microservicios, almacenar los datos a través de una API propia, implementar autenticación con roles y aislamiento entre organizaciones, y medir el consumo del sistema frente a los límites de la capa gratuita.

Para revisar la plataforma se crearon tres cuentas de prueba, todas con la contraseña Admin2026.A:

Tabla 1. Cuentas de prueba.

Correo	Rol	Organización	Uso
admin.norte@unab.cl	Administrador	campus-norte	Ve la Sala 101 y modifica la configuración
consulta.norte@unab.cl	Consulta	campus-norte	Ve la Sala 101 sin poder modificarla
admin.sur@unab.cl	Administrador	campus-sur	Comprueba que no ve datos de otra organización

Descripción del sistema

El sistema utiliza un sensor DHT11, que mide temperatura y humedad cada 10 segundos, y un sensor PIR, que detecta movimiento con 10 muestras por segundo. La Raspberry Pi combina ambas mediciones y genera un resumen por minuto con la siguiente información: el porcentaje del minuto con presencia, que determina si la sala está ocupada; un índice de confort de 0 a 100, que vale 100 entre 20 y 24 °C y entre 40 y 60 % de humedad; y un estado de la sala, por ejemplo "Ocupada - húmeda (ventilar)".

Además, el sistema genera alertas cuando la temperatura o la humedad superan sus umbrales y cuando se detecta presencia fuera del horario configurado. El sistema funciona las 24 horas; el horario configurable (por defecto de 8:00 a 22:00) solo se usa para la alerta de presencia fuera de horario. La plataforma web permite ver los datos en tiempo real, consultar el histórico, revisar los registros y las alertas, y administrar el dispositivo, cuya configuración se aplica sin reiniciar la Raspberry Pi.

Hardware y esquema de conexión

Se utilizó una Raspberry Pi 4 Model B. Aunque el enunciado pide al menos una Raspberry Pi 3, la Pi 4 cumple el requisito, ya que tiene el mismo cabezal GPIO de 40 pines y la misma arquitectura ARM64. Ambos sensores son digitales y entregan su señal a 3,3 V, que es el voltaje que soporta el GPIO, por lo que no fue necesario usar divisores de voltaje ni un conversor analógico. El PIR se alimenta con 5 V, pero su salida es de 3,3 V.

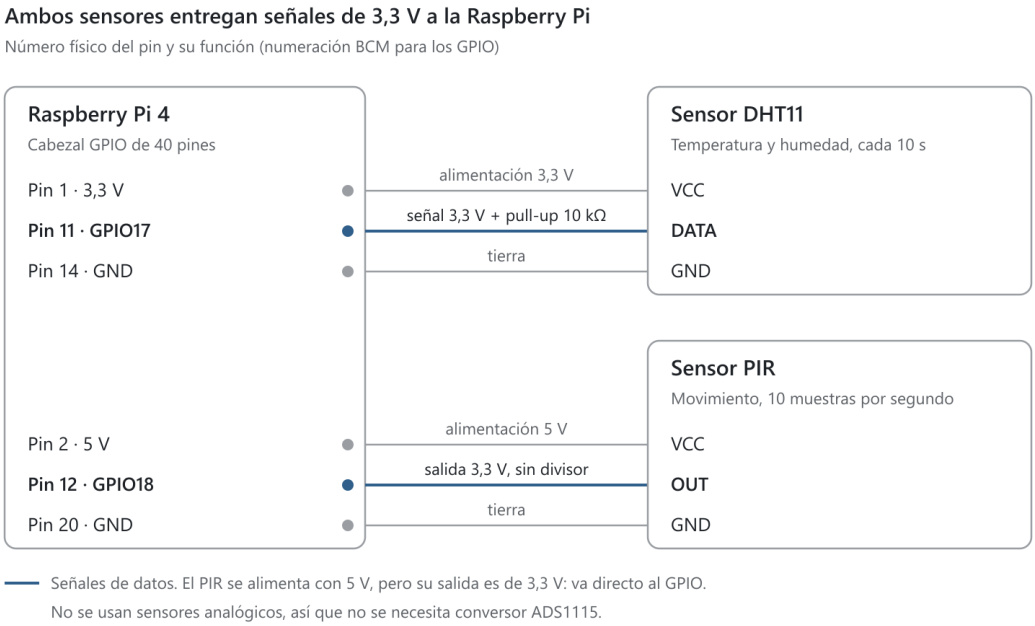


Figura 1. Esquema de conexión de los sensores.

Tabla 2. Pines utilizados.

Sensor	Pin del sensor	Raspberry Pi 4	Observación
DHT11	VCC	Pin 1 (3,3 V)	
DHT11	DATA	Pin 11 (GPIO17)	Pull-up de 10 kΩ si el módulo no la trae
DHT11	GND	Pin 14 (GND)	
PIR	VCC	Pin 2 (5 V)	El PIR necesita 5 V
PIR	OUT	Pin 12 (GPIO18)	Salida de 3,3 V, sin divisor
PIR	GND	Pin 20 (GND)	

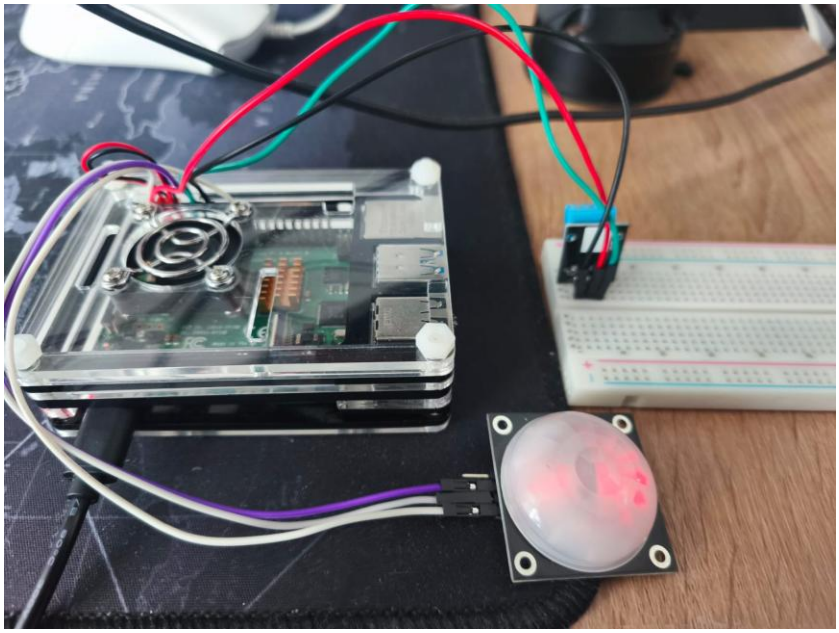


Figura 2. Montaje del sistema.

Arquitectura y microservicios

La arquitectura tiene tres partes: la Raspberry Pi, que captura y procesa los datos; la plataforma en Vercel, que contiene la API y el dashboard; y Cloud Firestore, donde se guardan los datos. Solo el servicio de sincronización se conecta a Internet y lo hace mediante conexiones salientes HTTPS, autenticándose con el token del dispositivo (Figura 3).

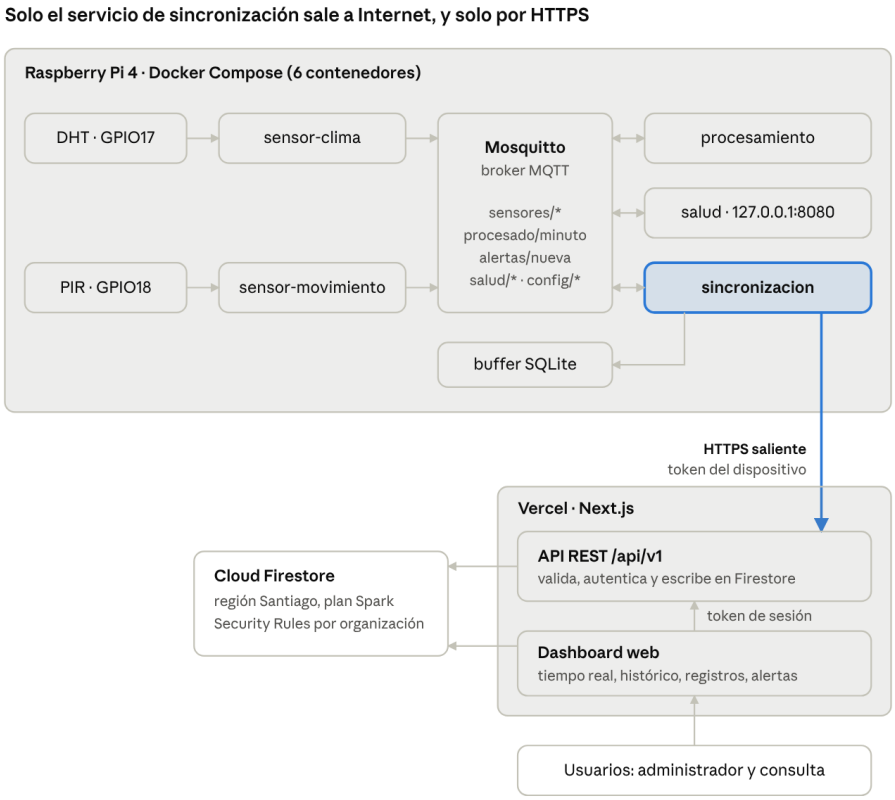


Figura 3. Arquitectura del sistema.

El software de la Raspberry Pi se dividió en seis microservicios, cada uno en su propio contenedor y orquestados con Docker Compose. Los servicios se comunican solo mediante mensajes MQTT y no comparten archivos ni bases de datos. La Tabla 3 resume la función de cada uno.

Tabla 3. Microservicios del dispositivo.

Servicio	Función
mosquitto	Broker MQTT que comunica a los demás servicios
sensor-clima	Lee el DHT11 cada 10 segundos y reintenta si la lectura falla
sensor-movimiento	Muestrea el PIR 10 veces por segundo y publica un resumen por segundo
procesamiento	Valida los datos, calcula el resumen por minuto y detecta alertas
sincronizacion	Guarda los resúmenes en SQLite, los envía a la API y obtiene la configuración
salud	Informa el estado de cada servicio, el uso de CPU y RAM y la temperatura de la placa

Se eligió esta división para que cada sensor pueda fallar o cambiarse sin afectar al resto, para que la lógica de cálculo se pueda probar sin hardware y para que una caída de la red solo afecte al servicio de sincronización. Todos los contenedores tienen reinicio automático (restart: unless-stopped) y

healthcheck, y se inician solos al encender la placa. Se usaron imágenes python:3.11-slim compatibles con ARM64, el GPIO se accede mapeando /dev/gpiochip0 sin modo privilegiado y el token del dispositivo se entrega mediante variables de entorno.

Si se pierde la conexión, los resúmenes quedan guardados en SQLite y se reenvían cuando vuelve la red. Cada resumen se identifica por su minuto, por lo que un reenvío sobrescribe el mismo documento en Firestore y no genera duplicados. Los datos solo se borran del buffer cuando la API confirma que los recibió.

Modelo de datos

Los datos se organizaron en Firestore con la siguiente estructura:

```
orgs/{orgId}
   devices/{deviceId}          (nombre, tokenHash, config, ultimaLectura, salud)
      lecturas/{minutoEpoch}   (ts, t, h, pres, mov, ocup, conf, estado)
      alertas/{minuto-TIPO}    (ts, tipo, estado, mensaje, valor)
```

Cada organización tiene su propia ruta, lo que permite aislarla con una sola regla de seguridad. La decisión más importante fue guardar un documento por minuto en lugar de uno por lectura: si se guardara cada medición, un dispositivo generaría cerca de 95.000 escrituras diarias (8.640 del DHT11 y 86.400 del PIR), mientras que con el resumen por minuto se generan 1.440. Además, la última lectura se copia en el documento del dispositivo, de modo que el panel en vivo obtiene el estado actual con una sola lectura. Como todas las consultas usan un solo campo (ts), no se necesitan índices compuestos, y se excluyeron de la indexación cinco campos que nunca se consultan para reducir el almacenamiento.

API REST

La API se desarrolló con Next.js y está desplegada en Vercel (https://raspberry-pi-y1p2.vercel.app/api/v1). Es el único componente que escribe en Firestore, valida todos los datos que recibe y verifica el token del dispositivo o la sesión del usuario antes de cada operación.

Tabla 4. Endpoints de la API.

Método	Ruta	Autenticación	Descripción
GET	/health	Ninguna	Verifica que la API esté disponible
POST	/ingest	Token del dispositivo	Recibe lecturas, alertas y estado de salud
GET	/device/config	Token del dispositivo	Entrega la configuración vigente
GET	/devices	Usuario	Lista los dispositivos de su organización
POST	/devices	Administrador	Crea un dispositivo y entrega su token
POST	/devices/{id}/token	Administrador	Genera un token nuevo y revoca el anterior
PUT	/devices/{id}/config	Administrador	Cambia intervalos, umbrales y sensores
GET	/readings	Usuario	Histórico por rango de fechas (máx. 31 días)

El código se organizó en tres capas: presentación (páginas, componentes y rutas de la API), lógica de negocio (autenticación, validación e ingesta, en lib/services) y acceso a datos (lib/data, único lugar que usa el Admin SDK de Firestore).

Seguridad

La Raspberry Pi no tiene credenciales de administrador de Firebase; estas existen solo como variables de entorno en Vercel. Cada dispositivo se autentica con un token propio de 192 bits, del cual Firestore guarda solo su hash SHA-256, y un administrador puede regenerarlo desde la web, lo que revoca el anterior. Los usuarios inician sesión con Firebase Authentication y tienen asignada una organización y un rol (administrador o consulta). Las Security Rules permiten leer solo los datos de la propia organización e impiden escribir desde el navegador. El dispositivo solo realiza conexiones salientes HTTPS y los archivos con secretos están excluidos del repositorio. El 21 de octubre se verificaron las reglas con solicitudes directas (Tabla 5).

Tabla 5. Verificación del control de acceso.

Prueba	Resultado
Leer datos de campus-norte sin sesión	403, denegado
admin.sur@unab.cl lee datos de campus-norte	403, denegado
consulta.norte@unab.cl lee datos de campus-norte	200, permitido
admin.norte@unab.cl escribe directo en Firestore	403, denegado
consulta.norte@unab.cl cambia la configuración por la API	403, requiere rol administrador

Despliegue

Las instrucciones completas se encuentran en el archivo docs/DESPLIEGUE.md del repositorio. En resumen, se crea un proyecto en Firebase (plan Spark) con Authentication y Firestore, se publican las reglas de seguridad, se importa el repositorio en Vercel (plan Hobby) con las variables de entorno indicadas en web/.env.example y se ejecuta el script npm run seed para crear las organizaciones, los usuarios y el dispositivo. En la Raspberry Pi se ejecuta raspberry/instalar.sh, se configuran API_URL y DEVICE_TOKEN en el archivo .env y se inician los servicios con docker compose up -d --build. Cada git push a la rama main vuelve a desplegar la plataforma automáticamente.

Evaluación de uso y capacidad

Escenario y estimación

Se consideró un dispositivo funcionando las 24 horas, que envía un resumen por minuto a la nube y consulta su configuración cada 30 segundos, y tres usuarios que usan el dashboard una hora al día y consultan dos veces el histórico de 24 horas. Cada documento pesa cerca de 0,6 KB contando los índices. La Tabla 6 muestra la estimación.

Tabla 6. Estimación de consumo diario.

Recurso	Cálculo	Por día	% del límite
Escrituras	1.440 lecturas + 1.440 actualizaciones del dispositivo + ≈ 10 alertas	2.890	14,5 % de 20.000
Lecturas	1.440 envíos + 2.880 configuraciones + ≈ 420 del panel + 8.640 del histórico	13.400	27 % de 50.000
Almacenamiento	1.440 documentos × 0,6 KB	0,86 MB	1 GiB dura ≈ 3,3 años
Invocaciones Vercel	1.440 envíos + 2.880 configuraciones + ≈ 20 de la web	4.340	13 % de 1.000.000 al mes

Se observa que el 65 % de las lecturas estimadas proviene del histórico, por lo que el consumo depende más del uso de la plataforma web que de los sensores.

Medición real

El sistema se midió entre el 21 y el 28 de octubre de 2026 con la consola de uso de Firebase y el panel Usage de Vercel (Anexo A). Los valores de Firebase corresponden al contador del día al momento de cada captura, y Vercel entrega un total acumulado, por lo que su valor diario se obtuvo como promedio.

Tabla 7. Consumo medido frente al estimado.

Día	Escrituras	Lecturas	Invocaciones Vercel
21-oct (parcial, desde 12:40)	134	1.100	257 acumuladas
22-oct	1.100	4.100	3.600 acumuladas
23-oct	1.400	2.100	—
24-oct	650	967	—
25-oct	653	974	—
26-oct	1.700	2.500	19.000 acumuladas
27-oct	1.800	2.700	27.000 acumuladas
Promedio (22 al 27-oct)	1.217	2.224	3.857 por día
Estimado	2.890	13.400	4.340 por día
Diferencia	−58 %	−83 %	−11 %

Las escrituras y lecturas medidas fueron menores que las estimadas. La principal razón es que la Raspberry Pi no estuvo encendida las 24 horas todos los días; por ejemplo, el 24 de octubre se registraron cerca de 650 escrituras, lo que indica que funcionó solo una parte del día. Sin embargo, la consola de Firestore muestra que el día de mayor actividad alcanzó 2.900 escrituras, prácticamente las 2.890 estimadas para un día completo, lo que confirma que el cálculo es correcto. En las lecturas la diferencia es mayor porque el histórico se consultó mucho menos de lo supuesto. En Vercel se registraron 27.000 invocaciones en siete días, solo un 11 % menos de lo estimado.

Límites de la capa gratuita

Tabla 8. Consumo de un dispositivo frente a los límites gratuitos.

Recurso	Límite gratuito	Uso estimado	Dispositivos que caben
Escrituras de Firestore	20.000 por día	2.890 por día	6
Lecturas de Firestore	50.000 por día	13.400 por día	9
Almacenamiento de Firestore	1 GiB	0,31 GB por año	10 dispositivos ≈ 4 meses
Invocaciones de Vercel	1.000.000 por mes	≈ 130.000 por mes	7

El primer límite que se alcanza son las escrituras diarias de Firestore, con siete dispositivos. Si se supera la cuota en el plan Spark, Firestore rechaza operaciones hasta el día siguiente sin generar cobros. El plan Hobby de Vercel solo permite uso no comercial.

Optimizaciones

Tabla 9. Decisiones que reducen el consumo.

Decisión	Antes	Después
Resumen por minuto en la Raspberry Pi	≈ 95.000 escrituras por día	1.440 (−98 %)
Alertas solo al cambiar de estado	hasta 5.760 escrituras por día	≈ 10
Última lectura copiada en el dispositivo	consulta de la colección	1 lectura
Panel en vivo limitado a 20 registros	60 lecturas al abrir	20 lecturas
Envío cada 5 min (propuesta)	2.880 escrituras por día	1.728 (−40 %)

Durante la medición el sistema operó con envío cada 60 segundos. En el día de mayor actividad se registraron 2.900 escrituras, unas 121 por hora, valor cercano a las 120 esperadas para esta configuración. El envío cada 5 minutos no se alcanzó a probar; según el cálculo, permitiría pasar de 6 a 11 dispositivos dentro de la capa gratuita.

Proyección de crecimiento

Para la proyección se usaron los precios de Firestore en la región de Santiago (USD 0,129 por 100.000 escrituras, USD 0,043 por 100.000 lecturas y USD 0,215 por GiB al mes) y el plan Pro de Vercel (USD 20 al mes con USD 20 de crédito y USD 0,60 por millón de invocaciones). Al costo de Firestore se le descuenta la cuota gratuita diaria, que se mantiene en el plan Blaze.

Tabla 10. Costo mensual estimado.

Dispositivos	Escrituras al mes	Firestore (USD)	Vercel (USD)	Total (USD)
1	86.700	0	0	0
10	867.000	0,83	20,00	20,83
100	8,67 millones	21,99	20,00	41,99
1.000	86,7 millones	233,54	77,76	311,30

El principal cuello de botella son las escrituras de Firestore, que con 1.000 dispositivos representan el 48 % del costo de la base de datos.

Costos de migración

Se compararon dos opciones PaaS y una IaaS, considerando 730 horas al mes y precios de pago por uso (Azure en East US y AWS en N. Virginia).

Tabla 11. Costo mensual de migración.

Alternativa	1 dispositivo (USD)	100 dispositivos (USD)	Administración
PaaS: Vercel Pro + Firebase Blaze	20,00	41,99	Baja
PaaS: Azure App Service + Firebase	12,41	46,81	Media
IaaS: AWS EC2 + PostgreSQL propio	16,16	32,33	Alta

Con 100 dispositivos la IaaS es cerca de USD 10 más barata al mes, pero el equipo tendría que encargarse de las actualizaciones del sistema operativo, los respaldos, los certificados TLS, el firewall y el escalamiento. En las opciones PaaS esas tareas las realiza el proveedor. Por esto se recomienda mantener Vercel Pro con Firebase Blaze, que no requiere cambiar el código.

Conclusiones

Se logró implementar un sistema IoT funcional que integra dos sensores, seis microservicios en contenedores, una API propia y una plataforma web con roles y organizaciones aisladas. El buffer local y los identificadores por minuto permiten que el sistema soporte cortes de red sin perder ni duplicar datos.

La evaluación de uso mostró que el sistema funciona sin costo y con margen: un dispositivo usa el 14,5 % de las escrituras y el 27 % de las lecturas gratuitas de Firestore. La decisión que más influyó fue agrupar las lecturas por minuto en la Raspberry Pi, que redujo las escrituras en un 98 %. La medición real confirmó el modelo, ya que el día de mayor actividad llegó a 2.900 escrituras frente a 2.890 estimadas, y las diferencias en el promedio se explican porque el dispositivo no estuvo encendido todo el tiempo y porque el histórico se consultó menos de lo previsto.

Si el sistema creciera, el primer límite serían las escrituras de Firestore, y con 1.000 dispositivos el costo llegaría a unos USD 311 al mes. Para ese escenario conviene mantener una arquitectura PaaS, ya que la diferencia de costo con una IaaS no compensa el trabajo de administrarla.

Referencias

Firebase. (2026). Precios de Firebase. https://firebase.google.com/pricing

Firebase. (2026). Cuotas y límites de Cloud Firestore. https://firebase.google.com/docs/firestore/quotas

Google Cloud. (2026). Precios de Firestore. https://cloud.google.com/firestore/pricing

Vercel. (2026). Precios. https://vercel.com/pricing

Vercel. (2026). Fair use guidelines. https://vercel.com/docs/limits/fair-use-guidelines

Microsoft. (2026). Precios de Azure App Service para Linux. https://azure.microsoft.com/pricing/details/app-service/linux/

Amazon Web Services. (2026). Precios de Amazon EC2 bajo demanda. https://aws.amazon.com/ec2/pricing/on-demand/

Amazon Web Services. (2026). Precios de Amazon EBS. https://aws.amazon.com/ebs/pricing/

Todas las fuentes fueron consultadas el 21 de octubre de 2026.

Anexo A. Capturas de uso

Capturas de la consola de uso de Firebase (contador del día) y del panel Usage de Vercel (acumulado de 30 días) durante el periodo de medición.

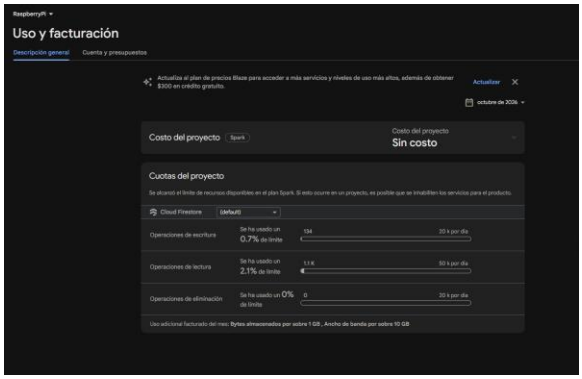


Figura A.1 Firebase, 21-oct: 134 escrituras, 1.100 lecturas

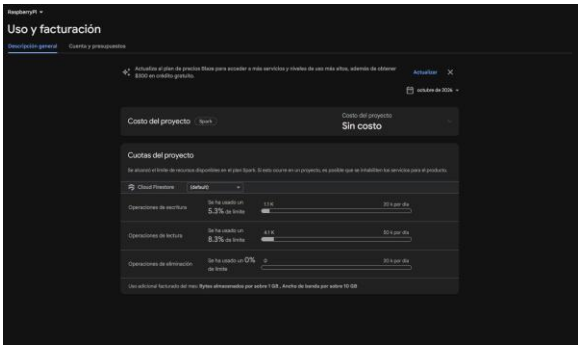


Figura A.2 Firebase, 22-oct: 1.100 escrituras, 4.100 lecturas

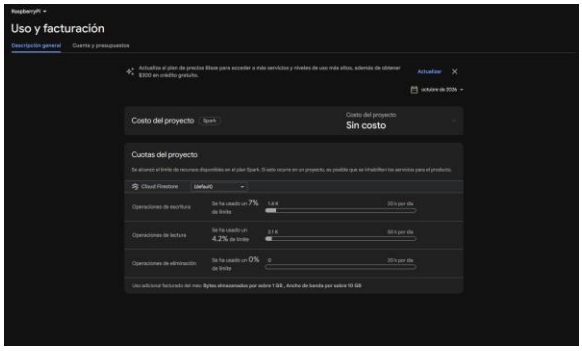


Figura A.3 Firebase, 23-oct: 1.400 escrituras, 2.100 lecturas

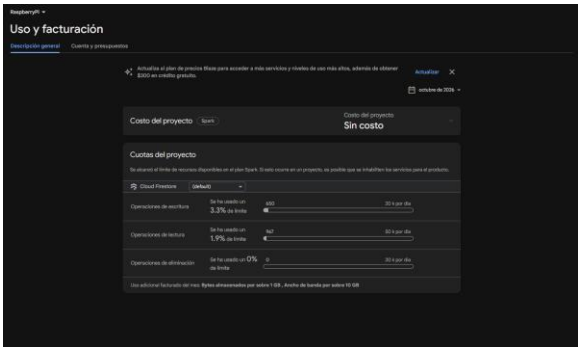


Figura A.4 Firebase, 24-oct: 650 escrituras, 967 lecturas

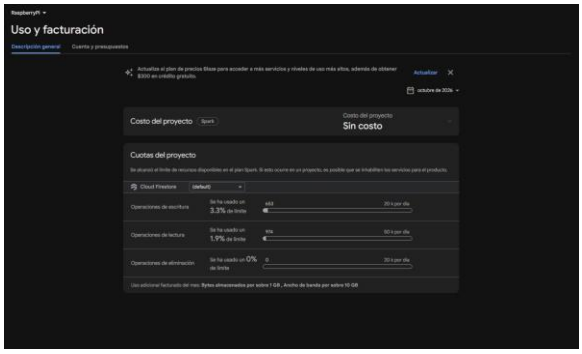


Figura A.5 Firebase, 25-oct: 653 escrituras, 974 lecturas

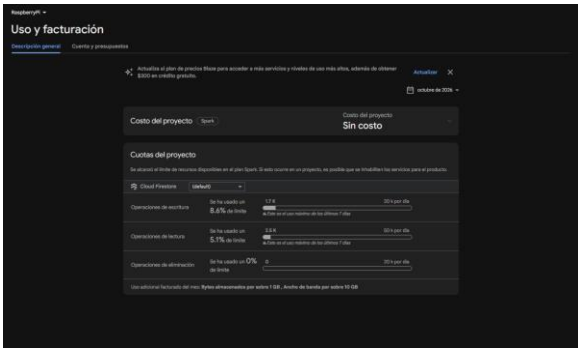


Figura A.6 Firebase, 26-oct: 1.700 escrituras, 2.500 lecturas

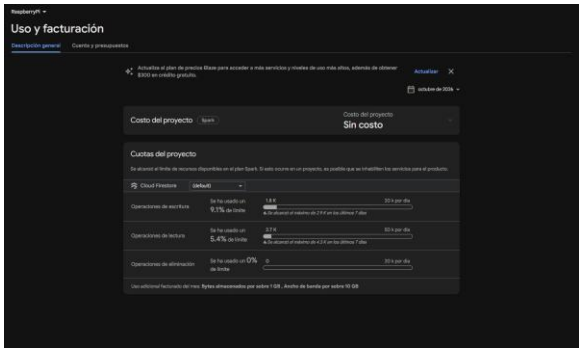


Figura A.7 Firebase, 27-oct: 1.800 escrituras; máximo semanal 2.900

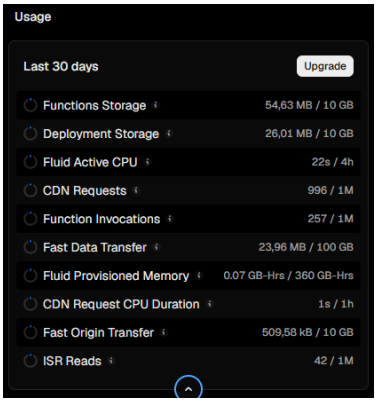


Figura A.8 Vercel, 21-oct: 257 invocaciones

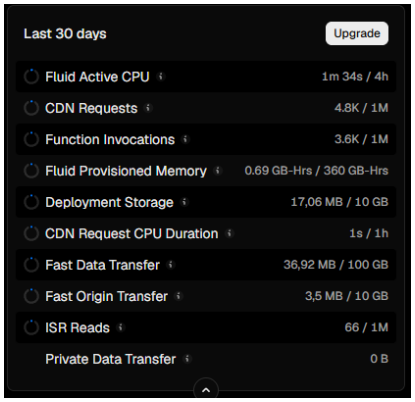


Figura A.9 Vercel, 22-oct: 3.600 invocaciones acumuladas

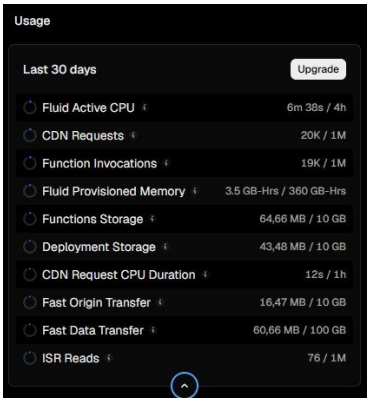


Figura A.10 Vercel, 26-oct: 19.000 invocaciones acumuladas

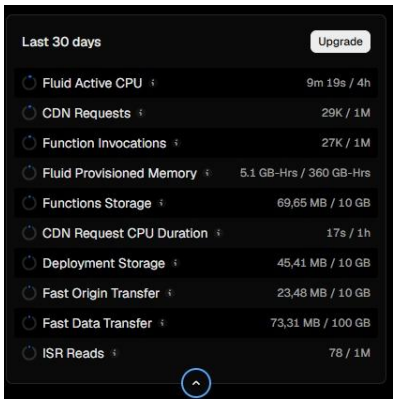


Figura A.11 Vercel, 27-oct: 27.000 invocaciones acumuladas